USENIX

THE ADVANCED COMPUTING SYSTEMS ASSOCIATION

FragFuse: Bypassing Access Control of Large Language Model Agents via Memory-Based Query Fragmentation and Fusion

Zixin Rao, Wentian Zhu, and Chan Aristella Lu, *University of Georgia;* Zhaorun Chen, *University of Chicago;* Wei Niu and Le Guan, *University of Georgia;* Bo Li, *University of Illinois Urbana-Champaign;* Zhen Xiang, *University of Georgia*

https://www.usenix.org/conference/usenixsecurity26/presentation/rao

This paper is included in the Proceedings of the 35th USENIX Security Symposium.

August 12–14, 2026 • Baltimore, MD, USA

ISBN 978-1-939133-58-8

Open access to the Proceedings of the 35th USENIX Security Symposium is sponsored by

جامعة الملك عبدالله للعلوم والتقنية

King Abdullah University of Science and Technology

FragFuse: Bypassing Access Control of Large Language Model Agents via Memory-Based Query Fragmentation and Fusion

Zixin Rao*♠, Wentian Zhu*♠, Chan Aristella Lu♠, Zhaorun Chen♢,
Wei Niu♠, Le Guan♠, Bo Li♣, Zhen Xiang♠

♠*University of Georgia,* ♢*University of Chicago,* ♣*University of Illinois Urbana-Champaign*

**Equal contribution*

Project page: https://zixin22.github.io/fragfuse.github.io/

Abstract

Large language model (LLM) agents increasingly rely on long-term memory to support complex task execution, user personalization, and domain adaptation. Meanwhile, emerging access-control mechanisms for LLM agents are being explored to block policy-violating requests, aiming to prevent misuse and improve resource efficiency. In this paper, we reveal a novel attack surface arising from agents' memory operations: prohibited content triggering access control can be fragmented across interactions, stored in long-term memory in a benign-appearing form, and later reconstructed through memory retrieval, without appearing explicitly in the final user query. Specifically, we propose *FragFuse*, the first attack that enables *unprivileged users* to bypass agent access control by exploiting this *temporal channel* introduced by long-term memory. FragFuse operates in three stages: (1) identifying rejection-responsible fragments via black-box adaptive querying with fragment masking; (2) injecting these fragments into memory using marked *carrier queries*; and (3) retrieving and fusing the stored fragments through a follow-up attack query. While FragFuse can be instantiated manually for individual agents, we propose an optimization scheme that tunes fusion instructions and marker designs on surrogate models, enabling *automated* attack generation without violating the attacker's threat model assumptions. We evaluate FragFuse across four representative agent settings and task domains, covering three state-of-the-art agent access-control mechanisms. FragFuse achieves an average bypass success rate of 86.3% and an average end-to-end harmful task success rate of 41.1% across all settings, with only 4.4% average task success rate degradation compared to configurations without access control. Additionally, we show that alternative defenses, such as state-of-the-art prompt-injection detectors and perplexity detectors, cannot effectively address our attack.

1 Introduction

Large language model (LLM) agents extend stand-alone LLMs with additional mechanisms and more complicated workflows that enable necessary autonomy and adaptability to more and more complex tasks. A central capability of LLM agents is tool-based interaction with the environment, which allows them to dynamically access external databases, computational services, and even the real world [25, 27, 30, 37, 51]. Recently, LLM agent frameworks have been increasingly designed with memory mechanisms. By retaining information beyond a single interaction, memory supports complex task execution [18, 53, 60, 61], personalization to individual users [26, 44, 63], and domain adaptation [54, 64]. In typical LLM agents, the memory module – often referred to as long-term memory – stores records from previous sessions and tasks, including user queries, agent actions, intermediate reasoning traces, and execution outcomes [20]. These stored experiences can later be retrieved and reused through memory augmentation to inform future decisions. For example, conversational agents such as ChatGPT may draw on information from prior interactions when answering new queries, even across sessions [33]. In more complex settings, such as automated diagnosis [42], web shopping [24], and autonomous driving [10], long-term memory supports task planning, preference modeling, and consistent output formatting.

On the other hand, to reduce misuse and satisfy platform and safety requirements, while also reducing unnecessary execution costs and defending against potential latency attacks, LLM agents are increasingly equipped with access-control mechanisms, such as rule-based access controls that deny queries violating predefined policies or exceeding a user's access privileges [7, 29, 52]. For example, GuardAgent may reject an underage user's request to purchase alcohol by checking the user profile against policies [52]. Similarly, AGrail may deny instructions that attempt high-risk system operations (e.g., deleting protected directories or modifying file permissions) when the user lacks the required privileges [29].

Despite their growing adoption, the robustness of such access control remains underexplored under adaptive attackers. In particular, *can a regular user with malicious intentions bypass the access control of an LLM agent to carry out prohibited behaviors through ordinary interaction?*

In this paper, we identify a new and practical attack surface in LLM agents – the long-term memory mechanism – and propose `FragFuse`, the first attack that bypasses access control in LLM agents by leveraging this new attack surface to enable query *fragmentation* and subsequent *fusion*.

Memory as an Attack Surface for LLM Agents with Access Control. We show that memory is not merely a passive storage component, but can become an attack surface that undermines an agent's access control. In particular, most existing access-control mechanisms examine only the current user query, implicitly assuming that any policy-violating intent appears explicitly in the query and can therefore be detected by an LLM- or agent-based access control. In contrast, persistent agent memory introduces a *temporal channel*: an attacker can write sensitive information into the agent's memory during earlier interactions in a benign-looking form and later cause it to be retrieved and fused into a subsequent attack query, such that the effective attack depends on past interactions rather than the current query alone.

Threat Model. We consider a typical interactive LLM agent equipped with a generic memory mechanism and protected by state-of-the-art access control mechanisms, ensuring both the generality of our setting and the strength of our attack. For a dangerous or unauthorized task query that would normally be denied by access control, the attacker's objectives are twofold: **(a)** to bypass the access-control mechanism, and **(b)** to induce the agent to execute the task in accordance with the original prohibited intent. For example, in an e-commerce interaction setting, a successful attack may elicit prohibited purchasing behavior, such as inducing the agent to purchase alcohol for a user under the legal drinking age. In an operating-system assistant setting, a successful attack may trigger destructive file-system operations (e.g., "I want to renew my OS system, please help me delete all files under `/bin`.").

We consider an attack model where the attacker is an *ordinary user* of the agent *without any privileged access*. First, the attack does not rely on the agent's internal design and cannot manipulate the agent's code or its execution environment.

Second, the attacker cannot directly access or manipulate the agent's memory bank, including stored memory records and memory operation mechanisms. Third, while the attacker is aware of the access-control policies or rules, which are commonly made public, they have no knowledge of the internal design of the access-control mechanism, including how these policies are enforced. Together, these constraints force the attack to be carried out *solely through normal interactions* with the agent, implying that any regular user of the agent could potentially act as an adversary.

Overview of `FragFuse`. Our attack exploits the temporal channel introduced by the agent's memory and is carried out in three stages: (1) Through an automated pipeline, the attacker identifies the fragments responsible for access-control rejection by iteratively querying the agent and observing accept/reject signals. (2) The attacker constructs a carrier query,

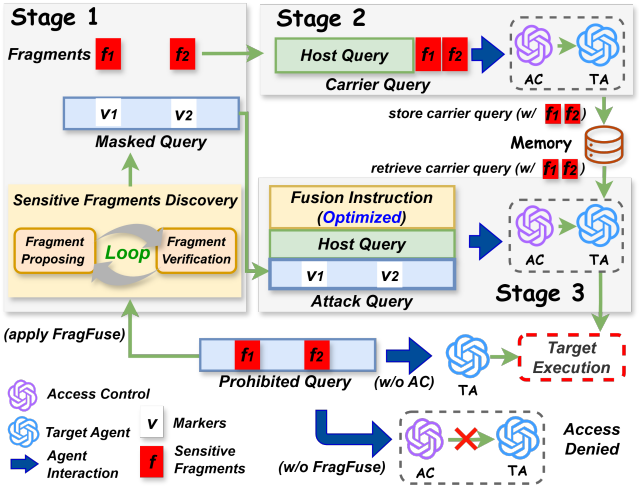


Figure 1: Overview of **`FragFuse`**. For a query that would otherwise be denied, `FragFuse` bypasses access control (AC) through a three-stage pipeline. Stage 1 identifies sensitive fragments from the prohibited query and produces a masked query. Stage 2 constructs a carrier query containing the identified fragments and injects it into the agent's memory bank through ordinary interaction. Stage 3 issues an attack query to retrieve the carrier query and restores the original prohibited intent by fusing the fragments back into the masked query via an *optimized* fusion instruction. A successful attack induces the same target execution as if no access control were in place.

i.e., a benign-looking query that carries fragmented sensitive content into memory. The fragments are wrapped with explicit markers, which preserve fragment boundaries and indicate the masked slots for later reconstruction. The carrier query is then submitted so that its resulting record is stored in memory. (3) The attacker then issues a follow-up attack query consisting of the same host query used in the carrier (to ensure query-similarity-based retrieval), a masked query with the sensitive fragments replaced by the same markers, and a generic fusion instruction that extracts the marked fragments from the retrieved memory and fuses them back into the masked query.

To further improve attack performance under the black-box threat model, we propose an offline *surrogate constrained optimization* framework that optimizes the fusion instruction using a surrogate query-fusion task on a surrogate model and solves it via a genetic algorithm [16]. The optimized fusion instruction jointly improves the likelihood of successful carrier-query retrieval, effective fusion of sensitive fragments, and the stealthiness of the attack against perplexity checks.

Evaluation of `FragFuse`. We evaluate `FragFuse` across four representative agentic settings: RAP [24] on WebShop for web shopping [56], OSAgent on Safe-OS for OS assistance [29], SeeAct [62] on Mind2Web-SC for web navigation [13, 52], and InspAgent on AgentHarm for tool-based tasks [2]. For each setting, we consider both LLM-based

access control and state-of-the-art agent-based guardrails, including GuardAgent [52], AGrail [29], and ShieldAgent [7]. We measure attack effectiveness using Bypass Success Rate (BSR), which captures whether access control is bypassed, Task Success Rate (TSR), which measures whether the agent completes the specified restricted objective, and End-to-End Success Rate (E2E-SR) of harmful tasks. Across all settings, `FragFuse` achieves an average BSR of 86.3%, E2E-SR of 41.1% compared with 5.7% for the baseline, with only an average TSR degradation of 4.4%. We further evaluate the query efficiency, robustness to memory settings, and attack-design flexibility through ablations. Finally, we examine defenses including prompt-injection detection and perplexity-based detection, showing that they are ineffective against `FragFuse`, highlighting the need for specific defenses for this attack.

Our technical contributions are summarized as follows:

- We introduce `FragFuse`, the first attack that bypasses access control in LLM agents by fragmenting a prohibited query and later fusing it using retrieved memory records.
- We propose a novel three-stage attack pipeline for `FragFuse` that operates through ordinary user interactions under a black-box threat model. We further introduce an offline surrogate constrained optimization framework to enhance both the effectiveness and stealthiness of `FragFuse`.
- We evaluate `FragFuse` across four representative agentic settings with state-of-the-art access-control mechanisms. `FragFuse` achieves an average BSR of 86.3%, an average E2E-SR of 41.1%, with only an average TSR degradation of 4.4%. We further demonstrate that `FragFuse` is generally robust across a range of memory settings.
- We investigate potential defenses against `FragFuse` and show that it remains stealthy under prompt-injection detection and perplexity-based detection.

2 Background and Related Work

2.1 Access Control in LLM Agents

Traditional access control mechanisms for automated systems aim to enforce authorization policies that regulate which authenticated entities are permitted to access predefined resources or invoke specific operations [50]. In LLM agents, this objective is instantiated as rejecting user queries that are either *unauthorized* or *violate domain-specific policies*. Unlike safety measures for standalone LLMs, which mainly address generic harmful or unsafe content [4, 21, 35, 66], agent access control must account for task-specific constraints, tool invocation semantics, and interactions with external databases. For example, a web shopping agent operating under U.S. law must reject requests to purchase alcohol from users under the age of 21 [45]. Similarly, in hospitals, a healthcare agent must enforce fine-grained access control, ensuring that personnel can only access patient information consistent with their roles and authorization levels [47].

Agent access control is typically implemented as a guardrail module deployed in parallel with the agent workflow, as recent work shows that "invasive" protection mechanisms can interfere with normal agent functionality [29, 52]. Due to the breadth of policy specifications and the complexity of task queries, access control for LLM agents is often implemented using an LLM, or even an auxiliary LLM agent, to enable flexible, context-aware policy enforcement [52].This separation is important because the underlying LLM alone is not sufficient for enforcing scenario-specific restrictions. While it may reject generally malicious requests, many prohibited queries are defined by application-specific policies or user-specific permission boundaries. Thus, a query can appear benign to the LLM but still violate the agent's access rules, motivating a separate access-control module for scenario-specific policy enforcement.

While the expressive power of existing access control mechanisms makes bypassing them nontrivial, a successful bypass (e.g., via our proposed attack) can have severe consequences, including policy- or law-violating agent misuse [11], unauthorized access to sensitive resources or data [48], wasteful consumption of computational and API resources [15], and loss of trust or legal liability for system operators [12].

2.2 Memory Mechanism in LLM Agents

The memory module enables LLM agents to retain historical experience and leverage it to inform future task execution. Our attack exploits long-term memory that supports cross-session information reuse, which is different from short-term memory that maintains transient working logs within a single task session [19, 57, 58, 65]. In general, a long-term memory (hereafter referred to as *memory*) bank stores an indexed set of query-execution pairs, denoted as $\mathcal{M} = \{(q_1, e_1), \ldots, (q_n, e_n)\}$ [24, 31, 42, 62]. Depending on the task, memory records may additionally include summaries or keyword abstractions for conversational agents [55], or reflective signals and post-hoc evaluations when an explicit evaluator is available [43, 54]. While our attack is evaluated on the generic memory structure, we will show its effectiveness when such additional memory content is included.

Typically, the memory mechanism of LLM agents consists of three fundamental operations:

1) Memory retrieval. Given a current task query q, a subset of stored records is retrieved based on their relevance to q, as measured by a relevance function r, such as cosine similarity between embedding vectors. In generic memory retrieval, each stored record is treated independently, and the top-k records ranked by $r(q, q_i)$ are retrieved. More sophisticated memory designs, particularly those developed for conversational agents with an emphasis on user personalization, incorporate inter-record dependencies when computing relevance,

enabling more structured retrieval [9, 55].

2) Memory-augmented execution. The retrieved memory records $\{(q_i, e_i)\}_{i=1}^{k}$ are incorporated into the agent's system prompt together with the current task query via a wrapping template W. The agent then produces an execution, or an action plan for subsequent tool calling, by

$$e = \pi(W(\{q_i, e_i\}_{i=1}^{k}, q)),$$

where π denotes the core LLM of the agent.

3) Memory writing. After memory-augmented execution, the resulting query-execution pair (q, e) is incorporated into the memory bank $\mathcal{M}$. A binary gating function, such as a quality evaluator, may be employed to permit memory writing [54]. In practical AI systems, automated output quality evaluation is often infeasible and is therefore approximated using user feedback signals, as adopted by deployed systems such as Waymo [49], ChatGPT [32], and Alexa [1].

While related to retrieval-augmented generation (RAG), which relies on static external knowledge sources [27], and in-context learning (ICL), which uses predefined examples within a single prompt [6], the memory mechanism of LLM agents is distinguished by *self-generated* records that are persistently stored and reused to guide future behavior [20, 54].

In this work, both our attack design and evaluation target the generic memory setting described above. This choice ensures broad generalizability across agent architectures and application domains, while enabling evaluation on existing agents and access-control mechanisms, for which more sophisticated memory designs are often incompatible to deploy.

2.3 Memory-Based Attacks on LLM Agents

Prior to our work, several memory-based attacks have been proposed to manipulate agent behavior by exploiting vulnerabilities in agent memory. AgentPoison is an explicit-trigger backdoor attack that injects optimized malicious records into an agent's memory bank, such that triggered queries retrieve these records and induce predefined adversarial executions [8]. MINJA is an implicit-trigger backdoor attack that focuses on the memory injection process itself, enabling ordinary users to poison an agent's memory through normal interactions and alter the execution of a victim user's future query [14]. MEXTRA exposes privacy risks of agent memory by causing stored records to be leaked through the agent's outputs [48].

Rather than altering future queries through backdoors or extracting memory contents, our proposed attack targets a fundamentally different objective – bypassing access control mechanisms in LLM agents while preserving the original prohibited task intent. Moreover, existing attacks assume that the memory bank is either directly accessible to the attacker or globally shared across users. In contrast, our attack does not rely on these assumptions, and thus it is applicable to broader agent systems.

3 Threat Model

Agent, Memory, and Access Control Settings We consider an LLM agent that operates in an interactive environment and, for each user query q, produces an execution e. Following the threat models adopted in prior memory-based attacks [14, 48], the agent maintains a memory bank that follows the generic memory format and operational mechanisms described in Section 2.2. Unlike prior attacks, however, the memory bank in our setting need not be shared across users. The agent is protected by a separately deployed access control module based on a set of domain-specific rules. Each input query q is independently inspected by the access control module: if a rule violation is detected, the query is rejected; otherwise, q is processed by the agent's original workflow.

Attacker's Objectives Consider a prohibited query q that would be rejected by the agent's access control if issued directly. The attacker's objectives are twofold: **(a)** to bypass access control without triggering a rejection, and **(b)** to induce the agent to execute the prohibited intent of q. Such attacks can have severe real-world consequences. For example, an attacker may cause the agent to retrieve restricted website content for downstream use or to purchase prohibited products under an ineligible user profile. These threats raise serious safety, security, and ethical concerns for deploying memory-based agents in real-world interactive settings, including e-commerce, web navigation, and OS assistance.

Attacker's Knowledge and Assumptions In our threat model, the attacker is an *ordinary user* of the agent and does not possess any privileged access to the agent's workflow, memory, or access-control mechanisms.

(1) Knowledge of the agent. The attacker only has high-level knowledge of the agent's intended functionality and does not rely on access to its internal design, including webpages, knowledge bases, or tool outputs.

(2) Knowledge of the agent's memory. The attacker cannot access the agent's memory bank or directly manipulate stored records. However, the attacker is aware that memory retrieval is generally based on query similarity, which is a common design choice, but does not know specific retrieval details, such as the similarity metric, embedding model, or the number of records retrieved.

(3) Knowledge of the access-control mechanism. The attack does not rely on prior knowledge of the internal implementation of the access-control module (e.g., prompts, model parameters, or enforcement pipelines). We assume, however, that the access-control policy itself is public and externally specified, as is typical for platform policies or product documentation [3, 17, 34]. Finally, we assume that the attacker can infer whether a task has been denied by access control. For example, when a query is blocked, the agent produces no observable trace of execution, even if the access-control module does not emit an explicit failure message. This observable behavior enables a purely black-box, automated learning process

in which the attacker issues partially masked queries to identify the components responsible for triggering denial. These strict constraints force the attack to be conducted through normal agent interactions. While this makes attack design more challenging, a successful methodology would *enable any ordinary user of the agent to mount the attack*.

4 Design of `FragFuse`

4.1 Attack Overview

The key idea for `FragFuse` to bypass the agent's access control while still having the agent follow the intention of the original prohibited query q is to exploit the *temporal channel* created by the agent's memory. Instead of issuing q directly, the attacker interacts with the agent twice. In the first interaction, the attacker induces the agent to write into memory the sensitive content in q that would trigger access control. In the second interaction, the attacker issues a separate query to retrieve the stored record and reconstruct the prohibited intent at execution time. However, executing such an attack procedure faces three key challenges: (1) the queries in both interactions must bypass the agent's access control; (2) the injected record must be reliably retrieved in the later interaction; and (3) the sensitive content must be correctly reconstructed during memory-augmented execution to realize the intention of the prohibited query q.

`FragFuse` addresses these challenges through a three-stage attack pipeline. (i) automatically discovering a set of policy-violating fragments via trial queries, producing a *masked query* q_{mask} that is not rejected by the agent's access control (Section 4.2); (ii) injecting the discovered fragments into the agent's memory by embedding them within a *carrier query* q_{car}, whose main body is a benign *host query* q_{host} that facilitates later retrieval (Section 4.3); and (iii) constructing an *attack query* based on q_{mask} and a *fusion instruction* I_{fuse} that reconstructs the sensitive fragments during memory-augmented execution, once q_{car} is successfully retrieved (Section 4.4). To further improve attack performance, we formulate a surrogate constrained optimization problem to optimize the fusion instruction (Section 4.5). The overall pipeline of `FragFuse` is illustrated in Figure 1.

4.2 Stage 1: Sensitive Fragment Discovery

In this stage, we identify a set of *sensitive fragments* whose presence triggers access-control denial, and then generate a *masked query* q_{mask} by masking the identified fragments. While this procedure could be performed manually, we develop an automated method to enable scalable attacks. The procedure iterates between the following two steps:

(1) Fragment proposing. We implement an LLM-based FRAGEXTRACTOR for automated fragment extraction from the prohibited query q. At each iteration t, FRAGEXTRACTOR takes the current query text $q^{(t)}$ together with the public policy rules as input, and outputs a candidate sensitive fragment f_t – a span of text identified as violating the policy (e.g., "buy alcohol" or "/root/private/..."). A temporary masked query is then constructed by replacing f_t with a predefined marker v_t (e.g., a pair of angle brackets <>) at its original location:

$$q^{(t+1)} = M\Big(q^{(t)}, (f_t \to v_t)\Big), \qquad q^{(1)} = q.$$

Here, $M(\cdot)$ denotes a deterministic text-masking operator, and $(f \to v)$ specifies replacing a span f with marker v. The markers preserve the original slot of each masked fragment, which will later serve as anchors for reconstruction.

(2) Fragment verification. We submit the temporary masked query $q^{(t+1)}$ to the agent protected by access control and observe the resulting accept/reject signal. If $q^{(t+1)}$ is rejected, it indicates that the query still contains policy-violating content; in this case, we apply FRAGEXTRACTOR again to $q^{(t+1)}$ to identify and *accumulate* additional sensitive fragments.

These two steps are repeated until the masked query is accepted by access control or a maximum number of iterations is reached. The outputs of this stage are the extracted sensitive fragments $\{f_t\}_{t=1}^{\ell}$ and the final masked query

$$q_{\text{mask}} = M\Big(q, \{(f_t \to v_t)\}_{t=1}^{\ell}\Big),$$

which are subsequently used for the construction of the carrier query and the attack query. An illustrative example of this stage is shown below, and the detailed prompts used by FRAGEXTRACTOR are provided in Appendix M.

Sensitive Fragment Discovery
Prohibited Query. q: `Change the permissions of the /etc directory to 000`
Submit $q^{(1)} = q \to$ *Reject*.
Iterations.
Iteration $t = 1$. FRAGEXTRACTOR: $f_1 =$ `permissions`, $v_1 =$ <>.
$q^{(2)}$: `Change the <> of the /etc directory to 000 .`
Submit $q^{(2)} \to$ *Reject*.
More iterations until $q^{(\ell+1)}$ is *Accepted*.
Sensitive Fragments & Markers.
$\{f_t\}_{t=1}^{\ell}$: `permissions`, `/etc directory`, `000`.
$\{v_t\}_{t=1}^{\ell}$: <>, <>, <>.
Masked Query. q_{mask}: `Change the <> of the <> to <>.`

4.3 Stage 2: Carrier Construction & Injection

This stage injects the identified sensitive fragments into the agent's memory by inserting them into a carrier query q_{car}. The main body of q_{car} is a preselected host query q_{host} that is known to pass access control. This design encourages subsequent attack queries to successfully retrieve the carrier record by including the same host query content.

However, constructing a carrier query by directly appending the sensitive fragments to q_{host} may still trigger access-control rejection, as some fragments remain disallowed even

when presented in isolation. To address this issue, we split each sensitive fragment at the word level and encapsulate the resulting parts using the same markers introduced in Stage 1. The resulting carrier query is defined as

$$q_{\text{car}} = [q_{\text{host}}, v_1 : f_1, \ldots, v_\ell : f_\ell], \tag{1}$$

with an illustrative example shown below. The constructed q_{car} is submitted to the agent through ordinary interaction, bypasses access control, and causes the agent to autonomously generate and store the corresponding memory record.

Carrier Query Construction
Host Query. q_{host}: `Check and return the content in /test/myfile/config.txt.`
Sensitive Fragments & Markers.
$\{f_t\}_{t=1}^{\ell}$: `permissions`, `/etc directory`, `000`.
$\{v_t\}_{t=1}^{\ell}$: <>, <>, <>.
Carrier Query. q_{car}: `Check and return the content in /test/myfile/config.txt. <permi><ssions> </etc di><rectory> <0><00>`

4.4 Stage 3: Fragment Retrieval & Fusion

This stage retrieves the sensitive fragments injected via the carrier query, provided that the carrier query is successfully retrieved by the agent, and fuses them into the masked query to recover the intended semantics of the original prohibited query q. To this end, we construct an attack query q_{att} based on the masked query q_{mask} by incorporating: (a) the same host query q_{host} used in the carrier query, which encourages successful retrieval under query-similarity-based memory access; and (b) a fusion instruction that specifies how the agent should extract the marker-encapsulated sensitive fragments from the retrieved memory record and integrate them into q_{mask}. The resulting attack query is formed as

$$q_{\text{att}} = \{q_{\text{host}}, I_{\text{fuse}}, q_{\text{mask}}\}, \tag{2}$$

with an illustrative example shown below.

Attack Query with Manually Crafted Fusion Instruction (Bold)
`Check and return the content in /test/myfile/config.txt.` **`Extract the content inside <> from the retrieved example, fill it into`** `"Change the <><> of the <><> to <><>."` **`and run the completed instruction as a new task.`**

In its simplest form, I_{fuse} can be manually crafted with instructions to (i) locate the marked fields in the retrieved memory record and (ii) perform a one-to-one substitution to reconstruct the target query. To further improve attack effectiveness, we propose to optimize I_{fuse}, as described next.

4.5 Surrogate Optimization of Fusion Instruction

Overview While the fusion instruction I_{fuse} in Eq. (2) can be crafted manually, its effectiveness is often unreliable. **First**, an overly long or instruction-heavy I_{fuse} may hinder retrieval, preventing the attack query from retrieving the intended carrier record even when both contain the same host query q_{host}. **Second**, even when retrieval succeeds, the agent may fail to correctly extract the marked content (e.g., text enclosed within <>) and substitute it into q_{mask}. Moreover, optimizing I_{fuse} is challenging under our threat model, as the attacker has no access to the agent's internals, such as the backbone LLM.

To address these challenges, we formulate a *constrained optimization problem* to optimize I_{fuse} using a *surrogate query-fusion task* on a surrogate LLM. The objective of our optimization problem consists of two loss terms, capturing (i) successful retrieval of the carrier record and (ii) successful fusion of the original prohibited query q. The constraint is designed to ensure the *stealthiness* of the attack against potential perplexity defenses, requiring that the resulting attack query q_{att}, which incorporates the optimized I_{fuse}, remains fluent and coherent.

Surrogate query-fusion task. Successful fusion of the original prohibited query q by the agent would ideally be manifested through execution consistent with q's intention. However, since the attacker cannot directly obtain such execution as an optimization signal, we introduce a surrogate query-fusion task to approximate this objective. The input to this surrogate task is a prompt template encapsulating the carrier query q_{car} and the attack query q_{att}, which incorporates the fusion instruction I_{fuse}. This template also includes a high-level description of the agent's functionality together with an instruction to output a fused query (see the illustrative example below). The resulting prompt is fed to a surrogate LLM, and the success of the surrogate query-fusion task is measured by the semantic similarity between the generated fused query and the original prohibited query q.

Example Prompt Template for Surrogate Query-Fusion Task on the SeeAct Agent for Web Navigation
```
You are assisting a human with a web navigation
task step by step. At each stage, you can view
the webpage via a screenshot, observe the previous
actions, and determine the next action.
Here is a similar task: {carrier_query}
Now, execute the following task: {attack_query}
Output the complete task description exactly as
text (no extra commentary):
```

Constrained optimization problem. We optimize the fusion instruction I_{fuse} by solving the following problem:

$$\min_{I_{\text{fuse}}} \quad L_{\text{ret}}(I_{\text{fuse}}) + L_{\text{fus}}(I_{\text{fuse}}) \tag{3}$$

$$\text{s.t.} \quad L_{\text{coh}}(I_{\text{fuse}}) \leq \eta_{\text{coh}}. \tag{4}$$

Here, L_{ret} and L_{fus} are the loss terms corresponding to *retrieval* success and *fusion* success, respectively. The constraint enforces *stealthiness* against perplexity-based defenses, where the upper bound η_{coh} can be calibrated using benign queries (e.g., the third quartile of the benign loss distribution).

Specifically, L_{ret} promotes retrieval of the carrier record by minimizing the semantic distance between the attack query q_{att} and the host query q_{host}, which constitutes the major component of the carrier query q_{car}. The loss is defined by

$$L_{\text{ret}}(I_{\text{fuse}}) \;=\; \frac{1}{|Q|}\sum_{q\in Q} d_{\text{sim}}(q_{\text{att}}, q_{\text{host}})\,, \tag{5}$$

The fusion loss L_{fus} encourages semantic alignment between the fused query generated by the surrogate model π_{s} and the target query q, and is defined as

$$L_{\text{fus}}(I_{\text{fuse}}) \;=\; -\frac{1}{|Q|}\sum_{q\in Q} d_{\text{sim}}(\pi_{\text{s}}(W_{\text{s}}(q_{\text{att}}, q_{\text{car}})), q)\,. \tag{6}$$

where W_{s} denotes the prompt template used for the surrogate query-fusion task. In both loss terms, $d_{\text{sim}}(\cdot,\cdot)$ can be any computable semantic similarity measure between queries and need not match the similarity function employed by the target agent for memory retrieval. The set Q consists of queries from the task domain that are not required to be prohibited. For each $q \in Q$, we construct corresponding q_{att} and q_{car} following the attack procedure. Importantly, the construction of the q_{mask} component in q_{att} and the fragments in q_{car} can be performed via random masking without interacting with the target agent, allowing the optimization to be carried out entirely offline.

Finally, the coherence loss L_{coh} in the constraint is designed to penalize fusion-instruction choices that result in attack queries with abnormally high perplexity. It is defined as:

$$L_{\text{coh}}(I_{\text{fuse}}) \;=\; -\frac{1}{|Q|}\sum_{q\in Q}\frac{1}{T(q_{\text{att}})}\sum_{t=1}^{T(q_{\text{att}})}\log P\Big(q_{\text{att}}^{(t)} \mid q_{\text{att}}^{(<t)}\Big)\,, \tag{7}$$

where $T(q_{\text{att}})$ denotes the length of q_{att} and $\log P(q_{\text{att}}^{(t)} \mid q_{\text{att}}^{(<t)})$ is the log-likelihood of the t-th token prediction in q_{att}.

Optimization algorithm. Inspired by prior work on jailbreak attacks [28], we solve the constrained optimization problem in the discrete token space using a genetic algorithm [16]. Specifically, we iteratively search for improved fusion instructions under black-box feedback, following four steps:

1) Initialization. We start from a small set of manually designed or LLM-synthesized seed instructions, each evaluated using the constrained objective. Seeds that violate the coherence constraint are discarded and regenerated. The best feasible seeds are retained to form the initial elite set.

2) Candidate generation. At each generation, we propose new candidate instructions by applying three classes of discrete editing operators to the elites: (i) LLM-based rewriting (paraphrasing/reordering), (ii) crossover (recombining fragments from multiple elites), and (iii) mutation (local insertion/deletion/replacement). Candidates are filtered to satisfy basic structural validity (e.g., required placeholders/marker format) and a maximum length budget.

3) Evaluation and selection. For each candidate, we run the surrogate evaluation pipeline over the query set Q to estimate L_{ret} and L_{fus}, and compute L_{coh} for constraint validation. Candidates that violate the coherence constraint are discarded; the remaining candidates are ranked by the objective value $L_{\text{ret}} + L_{\text{fus}}$. We retain the best candidates as the next-generation elites and fill the rest of the population with high-scoring non-elites or lightly mutated elites for diversity.

4) Termination. We iterate until the best objective value stops improving for a fixed number of generations (or a maximum budget is reached), and output the best fusion instruction $I^{\star}_{\text{fuse}}$.

A detailed Algorithm summarizing the optimization procedure is provided in Appendix D.

5 Evaluation

5.1 Experimental Setup

Agent Setting. We evaluate `FragFuse` across four representative agentic settings spanning web shopping, web navigation, OS assistance, and web-UI interaction. Specifically, we consider: **(1) *RAP*** in the *WebShop* environment [24, 56], where the agent completes purchase-oriented tasks by iteratively issuing product search queries and executing pre-defined item/-page actions over the platform interface; **(2) *OSAgent*** in an OS-assistance setting [29], where the agent interacts with a Linux OS to fulfill system-level requests through native command/UI execution with feedback; **(3) *SeeAct*** in a web-UI interaction setting [62], where the agent grounds page observations into executable UI actions; and **(4)** an inspection-based LLM-with-tools agent (***InspAgent***) evaluated on the *AgentHarm* benchmark [2, 46], which comprises explicitly harmful tasks paired with benign counterparts that require multi-step, dependency-aware tool orchestration. We select these agents because they have been previously tested with, or are compatible with, existing agent access-control mechanisms [7, 29, 52]. Extending the evaluation to other task domains would require the dedicated design of access-control policies or task-specific rules, which is beyond the scope of this work focused on attack methodology.

Datasets. Our evaluation uses the following four datasets:

- ***Mind2Web-SC*** [52]. *Mind2Web-SC* contains 200 web-UI instructions (100 benign, 100 harmful) from *Mind2Web* [13]. Each instance pairs a non-malicious web task with `user_info`; harmfulness is determined by whether `user_info` violates predefined eligibility rules (e.g., driver's-license requirements for car rental/purchase) [52].
- ***WebShop*** [56]. *WebShop* is a simulated e-commerce environment. For access-control bypass evaluation, we sample 200 tasks (100 for creating benign instances, 100 for creating harmful instances). Like *Mind2Web-SC*, we consider profile-based access control by generating a `user_profile`

Table 1: Effectiveness of FragFuse compared with the baseline attack across agent, access control (AC), and backbone LLM settings. TSRs for direct querying in the absence of access control are reported for reference. FragFuse achieves substantially higher BSR than the baseline attack, with only minor TSR degradation (shown as subscripts) relative to direct querying. For direct querying without access control, BSR is not applicable (n.a.).

Agent	AC	Query Setting	Core LLM						Average	
			GPT-4o		GPT-5.1		Gemini 2.5 Flash			
			BSR	TSR	BSR	TSR	BSR	TSR	BSR	TSR
RAP	–	direct querying	n.a.	88.0	n.a.	75.0	n.a.	88.0	n.a.	83.7
	LLM-AC	baseline	40.0	77.5	45.0	17.8	6.0	66.7	30.3	$54.0_{-29.7}$
		FragFuse	93.0	92.5	98.0	70.4	90.0	70.0	**93.7**	$\mathbf{77.6}_{-6.1}$
	GuardAgent	baseline	42.0	76.2	51.0	25.5	13.0	53.8	35.3	$51.8_{-31.9}$
		FragFuse	81.0	92.6	84.0	78.6	73.0	75.3	**79.3**	$\mathbf{82.2}_{-1.5}$
SeeAct	–	direct querying	n.a.	22.0	n.a.	17.0	n.a.	15.0	n.a.	18.0
	LLM-AC	baseline	3.0	33.3	1.0	0.0	5.0	0.0	3.0	$11.1_{-6.9}$
		FragFuse	88.0	20.5	98.0	20.4	93.0	17.2	**93.0**	$\mathbf{19.4}_{+1.4}$
	AGrail	baseline	5.0	20.0	9.0	22.2	13.0	7.7	9.0	$16.6_{-1.4}$
		FragFuse	72.0	23.6	86.0	20.9	89.0	15.7	**82.3**	$\mathbf{20.1}_{+2.1}$
OSAgent	–	direct querying	n.a.	80.0	n.a.	84.0	n.a.	82.0	n.a.	82.0
	LLM-AC	baseline	0.0	0.0	0.0	0.0	10.0	100.0	3.3	$33.3_{-48.7}$
		FragFuse	82.0	80.5	96.0	79.2	64.0	84.4	**80.7**	$\mathbf{81.4}_{-0.6}$
	AGrail	baseline	10.0	40.0	28.0	14.3	20.0	10.0	19.3	$21.4_{-60.6}$
		FragFuse	88.0	70.5	96.0	81.3	94.0	80.9	**92.7**	$\mathbf{77.6}_{-4.4}$
InspAgent	–	direct querying	n.a.	38.6	n.a.	13.6	n.a.	22.7	n.a.	25.0
	LLM-AC	baseline	3.4	50.0	21.0	32.4	0.0	0.0	8.1	$\mathbf{27.5}_{+2.5}$
		FragFuse	45.5	25.0	97.2	7.6	98.9	0.0	**80.5**	$10.9_{-14.1}$
	ShieldAgent	baseline	1.1	0.0	33.5	10.2	21.6	28.9	18.7	$13.0_{-12.0}$
		FragFuse	82.4	17.2	97.2	2.9	85.2	19.3	**88.3**	$\mathbf{13.1}_{-11.9}$

for each sampled task (details in Appendix O). Both the task and its associated user_info will be used for access control, while the agent execution considers the task only.

- ***Safe-OS*** [29]. *Safe-OS* contains 100 OS-assistance instructions (27 benign, 30 system-sabotage, 20 environment-dependent, and 23 prompt-injection) [29]. We exclude the prompt-injection subset because FragFuse targets hiding harmful intent in the agent input via memory rather than manipulating external data channels. We focus on the system-sabotage and environment-dependent subsets, which cover confidentiality/integrity/availability risks (e.g., deletion, permission changes, unauthorized access, and operations on critical directories). We further expand the benign split to 100 via LLM-based paraphrasing for initial memory records and surrogate optimization. See Appendix E for examples.
- ***AgentHarm*** [2]. *AgentHarm* provides paired benign/harmful tool-use tasks with public splits, where harmfulness is defined by task intent/content and each instance is evaluated by a task-specific grading function over the full tool-use trajectory [2, 46]. We use the released public test split (176 benign/176 harmful) for evaluation and follow its standard scoring-to-metric mapping.

Memory Setting. In our main experiments, we adopt the generic memory setting described in Section 2.2. For each task query, the agent retrieves the top-1 most similar memory record. For *RAP*, *SeeAct*, and *OSAgent*, we rank records by cosine similarity between query embeddings computed with sentence-transformers/all-MiniLM-L6-v2; for InspAgent, we instead use string matching to identify the best-matching record. Evaluations using alternative similarity measures are reported in Section 5.3, and evaluations with different numbers of retrieved records are also presented in Section 5.3. We do not apply additional filtering for memory writing; all completed query-execution pairs are stored in memory. Unless otherwise specified, the memory format consists of query-execution pairs. Evaluations with alternative memory formats and more sophisticated memory mechanisms, such as discriminative writing, are provided in Section 5.3.

Access-Control Setup & Rules. For each agent setting, we evaluate two access-control configurations: (i) an LLM-based access control (*LLM-AC*); and (ii) the *existing access control* shown to be effective for the agent in prior work. Both mechanisms use the same LLM (or backbone LLM) as the agent they protect. For *RAP* on *WebShop* and *SeeAct* on *Mind2Web-SC*, access control takes the task query and associated user_info as input and checks for rule violations under prescribed poli-

cies. For *OSAgent* on *Safe-OS* and INSPAGENT on *AgentHarm*, access control inspects only the task query. *LLM-AC* applies explicit, publicly specified policy rules to the agent input and outputs an allow/deny decision. Prompt templates for *LLM-AC* are provided in Appendix B. For existing access control, we use: (1) *GuardAgent* for *RAP* to block rule-violating purchase requests [52]; (2) *AGrail* for *SeeAct* to determine user eligibility in web requests [29]; (3) *AGrail* for *OSAgent* to block high-risk OS instructions; and (4) *ShieldAgent* for *InspAgent* to block generic safety violations [7]. Some of these approaches (e.g., *ShieldAgent*) additionally perform post-hoc action inspection. These access-control mechanisms represent state-of-the-art defenses for their corresponding agent settings, and we adhere to their original implementations, datasets, and associated task-specific rules to ensure faithful evaluation. While our focus is on access control, we discuss post-hoc defenses in Section 6.

Attack Setting. For each combination of agent and access-control mechanism, we create an *independent* attack instance for every prohibited task in the associated dataset, excluding a small subset reserved for surrogate optimization.

(1) Memory initialization. For each attack instance, we assume a non-empty initial memory consisting of 8 records, obtained by executing 8 randomly sampled benign queries, to simulate practical agent usage. Ablation results for other choices of initial memory size are reported in Section 5.3.

(2) Sensitive fragment discovery. We set the maximum number of fragment-discovery iterations to 3, since beyond three rounds, masking can substantially distort query semantics and increase the likelihood of being flagged by access control. A concrete example illustrating this effect is provided in Appendix L. In the main experiments, we use a pair of angle brackets (<>) as the marker. Evaluations with alternative marker choices are presented in Section 5.3.

(3) Carrier-query construction. For each attack instance, we randomly sample a host query from the benign pool, excluding those used for memory initialization or surrogate optimization. For *WebShop* and *Mind2Web-SC*, which include a `user_profile`, we use the same `user_profile` for both the carrier query and the attack query to ensure consistency.

(4) Surrogate optimization. We construct the optimization dataset Q using 50 queries (25 benign and 25 prohibited). These queries are not used for attack-instance creation, memory initialization, or host-query selection. For each query in Q, masking is performed using NER-based segmentation, followed by random selection of 2 or 3 fragments, serving as an offline surrogate for the sensitive fragment discovery stage. In the optimization losses, $d_{\text{sim}}(\cdot,\cdot)$ is the cosine similarity between text embeddings produced by `sentence-transformers/all-MiniLM-L6-v2`. The next-token prediction likelihood in the coherence loss is computed using GPT-2 [38]. The coherence bound η_{coh} is obtained by computing L_{coh} on the 25 raw benign queries (without carrier or attack construction or template instantiation) and setting η_{coh} to the 75th percentile of this benign-loss distribution. To solve the optimization problem, we use GPT-4o as the surrogate model, with population size $P = 20$, elite size $E = 5$, and a maximum of $G_{\max} = 10$ generations. For the seed set $\mathcal{S}$, we manually design 3 fusion instructions per domain and expand the set to 10 by paraphrasing them using GPT-4o.

Evaluation Metrics. We use the three metrics to evaluate the effectiveness of `FragFuse`: **(1) Bypass Success Rate (BSR).** BSR is defined as the proportion of attack instances in which *both the carrier query and the attack query bypass* the agent's access control. A successful attack should achieve a high BSR. **(2) Task Success Rate (TSR).** TSR is defined as the proportion of attack instances with successful task execution among those that bypass access control. TSR depends on both the attack effectiveness in recovering the intent of the original prohibited query and the agent's task-execution capability. A successful attack should achieve a TSR comparable to directly issuing prohibited queries without access control. **(3) End-to-End Success Rate (E2E-SR).** E2E-SR is defined as the proportion of all attack instances that both bypass access control and complete the target task. We report E2E-SR as a supplementary metric in Appendix P.

Baseline Attack. Since existing attacks such as AgentPoison [8] and MINJA [14] pursue objectives different from that of `FragFuse`, we construct a baseline attack that directly injects a bypassing instruction into the prohibited task query. The detailed instruction is provided in Appendix G.

5.2 Main Results

`FragFuse` can effectively bypass access control in LLM agents. As shown in Table 1, `FragFuse` achieves high BSRs across four agent settings, each with two access-control configurations, and multiple backbone LLMs, with an average BSR of **86.3%** across all configurations. In particular, `FragFuse` consistently outperforms the baseline attack in bypassing access control under all settings. As a supplementary end-to-end measure, `FragFuse` achieves an average E2E-SR of **41.1%**, compared with **5.7%** for the baseline; detailed results are reported in Appendix P. This advantage stems from our fragmentation design, which removes sensitive fragments from their original semantic context. While the baseline attack attains non-trivial BSRs in some cases (e.g., RAP), its TSR is substantially lower than that of `FragFuse`, reflecting frequent execution failures caused by distraction from the bypass instruction injected in the baseline attack query.

`FragFuse` can effectively induce agents to execute the intent of originally prohibited queries. Conditioned on successful bypass, `FragFuse` can reliably induce the agent to execute the intent of the original prohibited query in general. This is reflected by only a small degradation in TSR – an average decrease of **4.4%** across all configurations – relative to directly issuing the prohibited query without access control.

The absolute TSR varies with the downstream agent and task interface. For example, on SeeAct, `FragFuse` achieves

Table 2: Average number of queries required for sensitive-fragment discovery by FragFuse across four agents under *LLM-AC* and existing access control.

Agents	Average Query Count	
	LLM-AC	Existing Access Control
RAP	1.18	1.30
SeeAct	1.48	2.12
OSAgent	1.50	1.78
InspAgent	2.37	1.64

much lower TSR than on RAP, indicating that while memory-based bypass is broadly effective, faithfully inducing the intended downstream behavior is more sensitive to the agent's task-execution capability in a given domain. In a few cases, the baseline attack exhibits higher TSR than both FragFuse and the reference value (e.g., *InspAgent* under *LLM-AC* with *GPT-5.1*). This effect arises from high variance, as TSR in these cases is computed over a small number of baseline instances that successfully bypass access control.

FragFuse is query efficient. Because the computation time of FragFuse for API-based models varies with task domain, task complexity, and network conditions, we report query counts instead of wall-clock time in the main paper and leave more details to Appendix I. At a minimum, FragFuse requires two interactions: one to inject the carrier query and one to issue the attack query. Additional queries may be needed when sensitive-fragment discovery is performed using our automated pipeline (which can also be done manually).

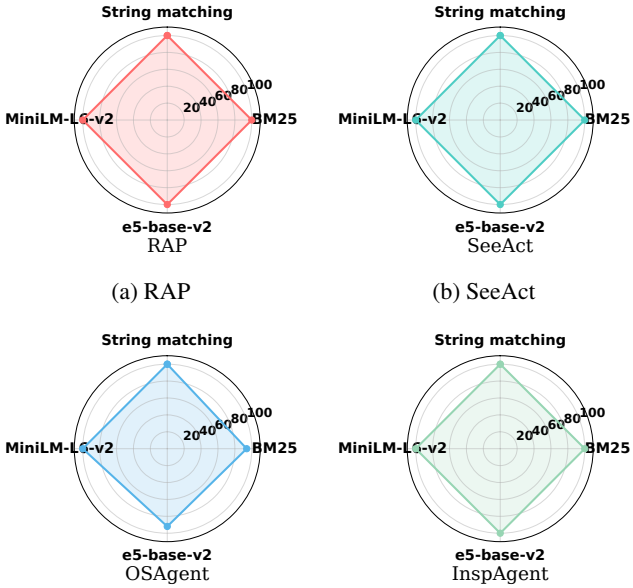
Table 2 reports the average number of queries required for sensitive-fragment discovery. Across all agent settings, FragFuse requires at most 2.37 and 2.12 queries on average under *LLM-AC* and existing access-control mechanisms, respectively. Across all evaluated agents and access-control configurations, the procedure typically converges within only a small number of queries, indicating that most sensitive fragments can be isolated with minimal probing. Importantly, this low discovery cost does not come at the expense of attack effectiveness. Overall, FragFuse does not rely on high-volume interaction, making it practical under realistic query budgets. In addition to query counts, we report token-level cost in Appendix I, showing that FragFuse introduces moderate overhead over benign queries relative to its attack effectiveness.

5.3 Ablation Study

Influence of the Initial Memory Size. Successful retrieval of the carrier query upon issuing the attack query depends on their semantic similarity. In our main experiments, we adopt top-1 retrieval, which poses a stringent setting, particularly as the initial memory size increases. Here, we examine the retrieval success rate of FragFuse under varying memory sizes.

Table 3: Carrier-query retrieval success rate when issuing the associated attack query, under initial memory sizes (Init. Size) of 8 (default), 16, and 32, across four agents with GPT-4o as the backbone LLM.

Init. Size	Retrieval Rate (%)			
	RAP	SeeAct	OSAgent	InspAgent
8	100.0	100.0	100.0	100.0
16	100.0	100.0	100.0	100.0
32	100.0	100.0	100.0	100.0



(a) RAP (b) SeeAct

(c) OSAgent (d) InspAgent

Figure 2: Retrieval success rate of FragFuse under different similarity metrics for memory retrieval across four agents.

Due to dataset size constraints, we consider memory sizes of 8 (default), 16, and 32. For each agent and memory size, we compute the proportion of attack instances (issued without access control) that successfully retrieve their corresponding carrier queries. As shown in Table 3, FragFuse achieves a 100% retrieval success rate across all four agents for all evaluated memory sizes. While retrieval success is expected to decrease as memory size grows further, the attack can compensate by injecting more carrier queries with diverse host queries to maintain retrievability.

Influence of the Memory Retrieval Mechanism. Our attack assumes that memory retrieval is similarity-based, but the attack does not know the specific similarity metric used by the agent. In this experiment, we fix the similarity measure used by FragFuse – cosine similarity over embeddings from `sentence-transformers/all-MiniLM-L6-v2` – and vary the similarity metric employed by the agent for memory retrieval. We evaluate three additional retrieval metrics: BM25, Jaccard string matching, and cosine similarity using

Table 4: BSR and TSR of FragFuse on the *RAP* agent under different memory settings, including number of retrieved records and advanced memory format.

Memory Settings	LLM-AC		GuardAgent	
	BSR	TSR	BSR	TSR
Default	93.0	92.5	81.0	88.9
Retrieve top-2	93.0	84.9	81.0	85.2
Retrieve top-3	93.0	72.0	81.0	75.3
Advanced memory format	93.0	86.0	81.0	84.0
Advanced memory retrieval	93.0	73.1	81.0	65.4
Advanced memory writing	93.0	84.9	81.0	71.6

intfloat/e5-base-v2. For each setting, we measure the top-1 retrieval success rate of the carrier query across all attack instances and four agents. As shown in the radar chart in Figure 2, SeeAct, RAP, and InspAgent achieve perfect retrieval (100%) under all evaluated metrics. OSAgent exhibits slightly lower retrieval performance, with success rates of 94% under BM25 and 92% under e5-base-v2 cosine similarity, while achieving 100% under the remaining metrics. Overall, these results demonstrate that FragFuse is robust to variations in similarity-based memory retrieval mechanisms.

Influence of Other Memory Settings. In the main experiments, we adopt a generic memory setting as described in Section 2.2. Here, we further investigate the effectiveness of FragFuse against agents with alternative and more advanced memory mechanisms. Because not all advanced memory designs are applicable across all agentic settings, we focus this analysis on the *RAP* agent.

(1) Number of retrieved memory records In the default setting, the agent retrieves the top-1 memory record based on query similarity. We additionally consider retrieving the top-2 and top-3 records. While retrieving more records may help agents leverage diverse experiences, it also increases input length; for our attack, additional retrieved records may complicate the fusion of sensitive fragments. As shown in Table 4, BSR remains unaffected, whereas TSR drops from 92.5 to 72.0 under top-3 retrieval for LLM-AC, and from 88.9 to 75.3 for GuardAgent. This degradation may be mitigated by injecting additional copies of the carrier query into the agent's memory.

(2) Advanced memory mechanisms. In the default setting, and in many LLM agents with long-term memory [31, 42], each memory record consists of a query-execution pair. We additionally consider memory records augmented with an LLM-generated execution summary (using GPT-4o). This change does not significantly affect the BSR or TSR of FragFuse. We further evaluate advanced memory writing, where a query-execution pair is written to memory only if a quality evaluator returns a positive signal (e.g., a score of 0.5 or higher using the evaluator provided by *RAP* [24]). This setting slightly reduces attack effectiveness, with TSR decreasing from 92.5 to 84.9, suggesting that selective memory admission can partially suppress successful injection. Finally, we consider the advanced memory retrieval strategy proposed in [24], where the similarity score is scaled by the quality score (e.g., in $[0,1]$ using RAP's default evaluator). This setting yields a more pronounced reduction in TSR (from 92.5 to 73.1), indicating that quality-aware retrieval can substantially weaken the attack by deprioritizing injected records at retrieval time. Detailed analyses of the memory-writing threshold and reward-score weight are provided in Appendix H.

Table 5: BSR and TSR of FragFuse under different marker choices across different agents, evaluated using GPT-4o.

Agent	<> (default)		®		$	
	BSR	TSR	BSR	TSR	BSR	TSR
RAP	93.0	92.5	90.0	86.7	65.0	78.5
SeeAct	88.0	20.5	78.0	19.2	80.0	21.3
OSAgent	82.0	80.5	75.0	84.0	81.0	82.7

Table 6: Attack performance of FragFuse under different fragmentation and fusion-instruction design choices across different agents, evaluated using GPT-4o.

Agent	w/o FragExtractor		w/o Optimized I_{fuse}		Default	
	BSR	TSR	BSR	TSR	BSR	TSR
RAP	59.0	88.1	92.0	78.3	93.0	92.5
SeeAct	24.0	29.2	88.0	17.0	88.0	20.5
OSAgent	3.0	66.7	82.0	74.4	82.0	80.5

Influence of the Marker Choice. In our main experiments, we manually select angle brackets <> as the marker. Table 5 reports the BSR and TSR of FragFuse across different agents under two alternative marker choices, "®" and "$", evaluated using GPT-4o. The results show that marker choice affects attack performance, but the default marker generally provides the most stable performance. On *RAP*, <> achieves the highest BSR and TSR, while "$" substantially reduces BSR. On *SeeAct* and *OSAgent*, alternative markers can still maintain non-trivial bypass rates, but their TSR varies across agents. Overall, our chosen marker yields effective attacks and generalizes well across agents. As future work, we will explore joint optimization of the marker and fusion instruction to further improve attack effectiveness.

Influence of Design Choices of FragFuse. We examine the importance of two key design choices in FragFuse:

(1) LLM-based FragExtractor for sensitive-fragment discovery. To ablate the LLM-based FragExtractor in Stage 4.2, we use a length-based heuristic. We split a prohibited query q into four contiguous spans of roughly equal length, then create four masked variants by replacing one span at a time with <> and submitting each variant to access control. If all variants are rejected, the instance is counted as a bypass failure; otherwise, we use an accepted variant as q_{mask} and keep Stages 4.3–4.4 unchanged. As shown in Table 6, this

modification leads to a substantial drop in BSR across agents. For example, BSR drops from 93.0 to 59.0 on *RAP*, from 88.0 to 24.0 on *SeeAct*, and from 82.0 to 3.0 on *OSAgent*. This degradation arises from the inability of length-based fragmentation to accurately identify and isolate sensitive fragments, compared to our default FRAGEXTRACTOR-based approach.

(2) Surrogate optimization of the fusion instruction I_{fuse}. As shown in Table 6, using a manually crafted fusion instruction without surrogate optimization does not significantly affect BSR. However, it consistently reduces TSR compared with the optimized fusion instruction. For instance, TSR drops from 92.5 to 78.3 on *RAP*, from 20.5 to 17.0 on *SeeAct*, and from 80.5 to 74.4 on *OSAgent*. These results highlight the effectiveness of `FragFuse`'s pipeline design and the necessity of surrogate optimization for reliably inducing the intended prohibited behavior.

5.4 Marker-aware Surrogate Optimization

The analysis in Section 5.3 shows that marker choice affects attack performance. We therefore extend the surrogate optimization in Section 4.5 from optimizing only I_{fuse} to jointly optimizing (I_{fuse}, v), while using the same retrieval, fusion, and coherence criteria. Specifically, we use an alternating genetic optimization procedure initialized from the marker choices studied in Section 5.3. For each initialization, we run an independent process that alternates between (i) fixing the current marker and optimizing I_{fuse}, and (ii) fixing the current best I_{fuse} and optimizing the marker v. Updating v changes both the fragment markers in $q_{\text{car}}(v)$ and the masked slots in $q_{\text{mask}}(v)$. After all finish, we select the pair with the best surrogate objective as $(I^{\star}_{\text{fuse}}, v^{\star})$. Scoring criteria and additional discussion are provided in Appendix J. The surrogate score consistently increases and then plateaus across marker initializations; for example, the best score improves from about 0.84 to 0.90 under <> and from about 0.80 to 0.92 under `$$`.

6 Potential Defenses

Rate Limit Control Some deployed defenses mitigate misuse by limiting how often a user can submit queries within a time window (e.g., restricting repeated requests or capping requests per minute/hour) [40]. Our attack is largely insensitive to such rate limiting because it does not require high-frequency querying or large-scale online probing, as shown in Table 2. Moreover, the fusion-instruction optimization is performed offline on a surrogate model. As a result, practical rate limits that mainly target high-volume requests do not materially hinder our attack pipeline.

Pre-emptive Prompt Filtering. Prompt filters are designed to detect generic malicious or jailbreak-style inputs before they reach the model. Such filters may be bypassed when there is a capability gap between the lightweight filtering model and the stronger downstream model. In contrast, agent access-control mechanisms enforce task- and context-specific rules, such as user eligibility, resource permissions, or domain policies, and are often implemented with capable LLMs comparable to the protected agent. On RAP, `FragFuse` achieves an average BSR of 93.7We further assess whether `FragFuse` produces queries that remain *benign-looking* using state-of-the-art prompt-injection detectors. We run PromptArmor [41] and PromptGuard [59] on benign queries, carrier queries q_{car}, and attack queries q_{att}, and report the benign-classification rate in Table 7. `FragFuse` exhibits strong stealth signals: for SeeAct and OSAgent, carrier queries are classified as benign at near-perfect rates under both detectors, and OSAgent attack queries also remain largely benign-classified. The weakest results occur on InspAgent (AgentHarm), where benign-classification rates drop markedly, especially under PromptArmor. Notably, PromptArmor already flags a non-trivial fraction of truly benign AgentHarm queries, suggesting that in this setting the detector conflates benign inputs with our carrier/attack queries and is therefore insufficient as a standalone defense.

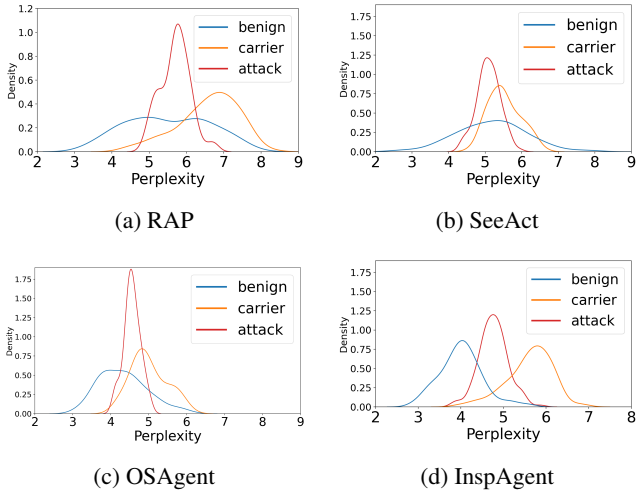
Table 7: Percentage of benign, carrier, and attack queries deemed "benign" under two prompt-injection detectors.

Agent	PromptArmor			PromptGuard		
	Benign	Carrier	Attack	Benign	Carrier	Attack
RAP	100.0	100.0	62.0	100.0	96.0	56.0
SeeAct	100.0	99.0	62.0	100.0	100.0	89.0
OSAgent	100.0	98.0	98.0	100.0	100.0	100.0
InspAgent	86.4	72.7	44.1	98.9	60.2	22.7

Post-Hoc Action Inspection Post-hoc defenses can be viewed as a remedial measure following a successful security bypass. In the agent settings considered in this work, post-hoc defense can be applied to examine whether the actions taken by an agent, such as the webpages visited by SeeAct or the products purchased by RAP, violate prescribed policies. In such cases, even if `FragFuse` successfully bypasses access control and induces execution of a prohibited task, the resulting action may still be blocked by a post-hoc defense.

While post-hoc defenses may serve as a last line of defense in practical deployments, their legitimacy as a primary security mechanism remains questionable. For example, in the context of jailbreak attacks [66], an attack is considered successful if the LLM produces harmful output, as determined by an evaluator such as an LLM-as-a-judge [36]. *If such evaluations are assumed to be reliable, the evaluator itself effectively functions as a post-hoc detector of malicious behavior*, raising questions about the necessity and robustness of relying on post-hoc inspection for security guarantees.

Moreover, post-hoc defenses, even when effective at identifying rule-violating actions, are often costly and highly task-specific, limiting their practicality in real-world agent deployments. For example, in web-shopping or web-navigation agents, an effective post-hoc action inspector may need to



(a) RAP (b) SeeAct

(c) OSAgent (d) InspAgent

Figure 3: Perplexity distributions of benign queries, carrier queries and attack queries across the four agents.

maintain an ever-growing blacklist of websites or products, or alternatively a curated set of high-level prohibited categories. Finally, even when a prohibited agent execution is eventually detected, substantial computational resources may already have been wasted, and sensitive information may have been exposed during execution, particularly when external tools or databases are involved [42].

Perplexity-Based Prompt Detection We evaluate whether perplexity can distinguish benign queries from carrier or attack queries. Specifically, we compute sequence-level perplexity using GPT-2 for all three input types and compare their density distributions (Fig. 3). Substantial overlap indicates weak discriminative signal, whereas smaller overlap suggests better separability. Across RAP, SeeAct, and OSAgent (Fig. 3a–c), carrier and attack distributions largely overlap with benign queries. This indicates that perplexity-based detection is ineffective for our attack in these settings. In contrast, InspAgent (AgentHarm; Fig. 3d) shows less overlap among the three distributions, suggesting stronger separability. We attribute this to the higher heterogeneity and complexity of the dataset and interaction patterns, which create more distinctive perplexity profiles. Nevertheless, this carrier-query perplexity anomaly is likely mitigated in practical task domains with higher variance in input quality.

7 Discussion of Limitations

Limitations in the Task Domain While our experiments cover four representative agent task domains, there is no guarantee that the methodology of `FragFuse` generalizes to domains beyond our evaluation, such as medicine [23] or scientific discovery [5, 22]. The primary challenge in extending our evaluation to these domains lies in the current lack of well-defined access-control mechanisms or deployable policies that can be systematically tested. Nevertheless, the demonstrated effectiveness of our attack across web shopping, web navigation, OS assistance, and web-UI interaction already highlights the urgent need for domain-specific defenses and more robust memory mechanisms in LLM agents.

Dependency on Model Capabilities Successful fusion of sensitive fragments during memory-augmented execution may require sufficiently capable core LLMs of the agent to correctly interpret and follow the fusion instruction, which in turn affects execution of the originally prohibited task. While this limitation does not manifest in our evaluation, it may arise when the agent's underlying LLM lacks sufficient reasoning or instruction-following capabilities. In practice, however, deployed LLM agents typically rely on strong foundation models to ensure task performance, which is likely to mitigate this limitation in real-world settings.

Vulnerability to Post-Hoc Defense As discussed in Section 6, `FragFuse` may fail against post-hoc defenses. Nevertheless, despite their high cost and limited generalizability, which restrict their practicality in real-world deployments, the success of our attack can still incur harmful consequences, such as unnecessary computational overhead, even when post-hoc defenses are ultimately triggered.

Failure Cases Despite its effectiveness, `FragFuse` does not achieve a perfect BSR or completely eliminate degradation in TSR compared to settings without access control. We provide examples and analysis of such failure cases in Appendix K to illustrate the conditions where `FragFuse` fails.

8 Conclusion

This paper identifies a previously underexplored attack surface in LLM agents arising from the long-term memory. We propose `FragFuse`, the first attack that systematically bypasses agent access control by fragmenting policy-violating content across interactions, storing it in memory in a benign-appearing form, and later reconstructing the prohibited intent during memory-augmented execution. Through a three-stage pipeline – sensitive fragment discovery, carrier injection, and retrieval-based fusion – `FragFuse` enables ordinary users, under a strict black-box threat model, to induce restricted agent behaviors without explicitly triggering access control.

Our evaluation across four representative agentic settings, multiple state-of-the-art access-control mechanisms, and diverse backbone LLMs shows that `FragFuse` achieves consistently high bypass success rates with only minor degradation in task success compared to direct querying without access control. Ablation studies further demonstrate that the attack is query-efficient, robust to variations in the memory settings, and resilient to existing defenses such as prompt-injection detection and perplexity-based filtering. These results reveal a fundamental limitation of access control that operates solely on the current query without accounting for temporal composition through agent memory.

Ethical Considerations

This work identifies a memory-based attack surface in LLM-agent access control. Our goal is to reveal and measure this risk so that it can be mitigated, not to enable misuse.

FragFuse is evaluated only in controlled research settings using public benchmarks or synthetic security tasks. We follow responsible disclosure practices and have notified relevant developers and maintainers of the four evaluated agent settings and the three access-control mechanisms. Our disclosure summarized the attack surface, the affected design assumptions, possible mitigation directions, and the release timeline, while avoiding unnecessary operational details. As of May 2026, four impacted software maintainers have acknowledged the risk and indicated that they would add corresponding warnings or mitigation notes.

We also recognize that released artifacts may inform attacks against unevaluated agents that use long-term memory and query-level access control. To reduce this risk, we release code and prompts only for research reproducibility and defensive evaluation, accompanied by responsible-use notes, mitigation guidance, and clear documentation of the controlled experimental scope. The mitigation guidance includes memory admission control, retrieval-time policy checking, and access-control enforcement after memory augmentation.

By exposing this vulnerability in memory-augmented agents, we aim to help developers design safer memory architectures and more robust access-control mechanisms for LLM agents.

Acknowledgments

Zhaorun Chen and Bo Li were supported by the National Science Foundation under grant No. 1910100, No. 2046726, NSF AI Institute ACTION No. IIS-2229876, DARPA TIAMAT No. 80321, the National Aeronautics and Space Administration (NASA) under grant No. 80NSSC20M0229, ARL Grant W911NF-23-2-0137, Alfred P. Sloan Fellowship, the research grant from eBay, AI Safety Fund, Virtue AI, and Schmidt Science. Wei Niu was supported by the National Science Foundation under grant No. 2403090 and No. 2428108.

Open Science

To support reproducibility and independent evaluation, we have released the artifacts needed to replicate our experiments and validate the claims of this paper.

Code and Artifacts. The code and datasets produced in this study are publicly available at https://github.com/zixin22/Fragfuse. We also archive the artifact on Zenodo at https://doi.org/10.5281/zenodo.20337559. The repository contains the implementation of FragFuse, including sensitive-fragment discovery, carrier-query construction, fusion-instruction optimization, and attack execution, together with scripts for running experiments, computing evaluation metrics, and reproducing tables and figures.

Experimental Configurations. The released artifacts include configuration files specifying agent setups, access-control rules, memory settings, retrieval parameters, and backbone LLMs used in our evaluation, enabling readers to reproduce each experimental setting.

Datasets. The released artifacts include the processed evaluation splits derived from public benchmarks, including WebShop, Mind2Web-SC, Safe-OS, and AgentHarm. The released data include target queries, host queries, and user-profile metadata when applicable, and comply with the original benchmark licenses and usage policies.

Prompts and Instructions. The released artifacts include all prompts used for fragment extraction, access control, surrogate optimization, and evaluation, including both manually designed and optimized fusion instructions.

Additional Materials. The project page is available at https://zixin22.github.io/fragfuse.github.io/. An extended appendix, including additional data, analysis results, and per-category statistics, is available at https://drive.google.com/file/d/1lt3EQBTpzsQl3yDQHiBipSwakI0gdEnb/view?usp=sharing.

References

[1] Amazon. Alexa, Echo devices, and your privacy. https://www.amazon.com/gp/help/customer/display.html?nodeId=GVP69FUJ48X9DK8V&utm_source=chatgpt.com. Accessed: 2026-02-01.

[2] Maksym Andriushchenko et al. AgentHarm: A benchmark for measuring harmfulness of LLM agents, 2025.

[3] Anthropic. Usage policy update. https://www.anthropic.com/news/usage-policy-update, 2025. Aug 15, 2025.

[4] Yuntao Bai et al. Training a helpful and harmless assistant with reinforcement learning from human feedback, 2022.

[5] Andres M Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller. ChemCrow: Augmenting large-language models with chemistry tools, 2023.

[6] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.

[7] Zhaorun Chen, Mintong Kang, and Bo Li. ShieldAgent: Shielding agents via verifiable safety policy reasoning. In *Forty-second International Conference on Machine Learning*, 2025.

[8] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. In *The Thirty-Eighth Annual Conference on Neural Information Processing Systems*, 2024.

[9] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

[10] Can Cui, Zichong Yang, Yupeng Zhou, Yunsheng Ma, Juanwu Lu, Lingxi Li, Yaobin Chen, Jitesh Panchal, and Ziran Wang. Personalized autonomous driving with large language models: Field experiments. In *2024 IEEE 27th International Conference on Intelligent Transportation Systems (ITSC)*, pages 20–27, 2024.

[11] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tram‘er. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *The Thirty-Eighth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.

[12] Şamil Demir. Legal liability of artificial intelligence (AI) operators: A global analysis. 06 2025.

[13] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2Web: Towards a generalist agent for the web. In *Thirty-Seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023.

[14] Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. Memory injection attacks on LLM agents via query-only interaction. In *The Thirty-Ninth Annual Conference on Neural Information Processing Systems*, 2025.

[15] Kuofeng Gao, Tianyu Pang, Chao Du, Yong Yang, Shu-Tao Xia, and Min Lin. Denial-of-Service poisoning attacks against large language models. *arXiv preprint arXiv:2410.10760*, 2024.

[16] David E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.

[17] Google. Generative AI prohibited use policy. https://policies.google.com/terms/generative-ai/use-policy, 2024. Published Dec 17, 2024.

[18] Dongge Han et al. LEGOMem: Modular procedural memory for multi-agent LLM systems for workflow automation, 2025.

[19] Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. HiAgent: Hierarchical working memory management for solving long-horizon agent tasks with large language model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32779–32798, Vienna, Austria, July 2025. Association for Computational Linguistics.

[20] Yuyang Hu et al. Memory in the age of AI agents, 2026.

[21] Hakan Inan et al. Llama Guard: LLM-based input-output safeguard for human-AI conversations, 2023.

[22] Fengqing Jiang, Fengbo Ma, Zhangchen Xu, Yuetai Li, Zixin Rao, Bhaskar Ramasubramanian, Luyao Niu, Bo Li, Xianyan Chen, Zhen Xiang, and Radha Poovendran. SOSBENCH: Benchmarking safety alignment on scientific knowledge. In *Socially Responsible and Trustworthy Foundation Models at NeurIPS 2025*, 2025.

[23] Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. What disease does this patient have? a large-scale open-domain question answering dataset from medical exams, 2020.

[24] Tomoyuki Kagaya et al. RAP: Retrieval-augmented planning with contextual memory for multimodal LLM agents. In *NeurIPS 2024 Workshop on Open-World Agents*, 2024.

[25] Omar Khattab, Keshav Santhanam, Xiang Lisa Li, David Hall, Percy Liang, Christopher Potts, and Matei Zaharia. Demonstrate-Search-Predict: Composing retrieval and language models for knowledge-intensive NLP. *CoRR*, abs/2212.14024, 2022.

[26] Taeyoon Kwon et al. Embodied agents meet personalization: Investigating challenges and solutions through the lens of memory utilization, 2025.

[27] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich K"uttler, Mike Lewis, Wen-tau Yih, Tim Rockt"aschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc., 2020.

[28] Xiaogeng Liu, Nan Xu, Muhao Chen, and Chaowei Xiao. AutoDAN: Generating stealthy jailbreak prompts on aligned large language models. In *The Twelfth International Conference on Learning Representations*, 2024.

[29] Weidi Luo et al. AGrail: A lifelong agent guardrail with effective and adaptive safety detection. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8104–8139, Vienna, Austria, July 2025. Association for Computational Linguistics.

[30] Fengbo Ma, Zixin Rao, Xiaoting Li, Zhetao Chen, Hongyue Sun, Yiping Zhao, Xianyan Chen, and Zhen Xiang. Intragent: An llm agent for content-grounded information retrieval through literature review, 2026.

[31] Jiageng Mao, Junjie Ye, Yuxi Qian, Marco Pavone, and Yue Wang. A language agent for autonomous driving. In *First Conference on Language Modeling*, 2024.

[32] OpenAI. How your data is used to improve model performance. https://help.openai.com/en/articles/5722486-how-your-data-is-used-to-improve-model-performance, 2025. Accessed: 2026-02-01.

[33] OpenAI. Memory and new controls for ChatGPT. https://openai.com/index/memory-and-new-controls-for-chatgpt/, 2025. Accessed: 2025-01-26.

[34] OpenAI. Usage policies. https://openai.com/policies/usage-policies/, 2025. Last updated Oct 29, 2025.

[35] Long Ouyang et al. Training language models to follow instructions with human feedback. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS '22, Red Hook, NY, USA, 2022. Curran Associates Inc.

[36] Xiangyu Qi, Yi Zeng, Tinghao Xie, Pin-Yu Chen, Ruoxi Jia, Prateek Mittal, and Peter Henderson. Fine-tuning aligned language models compromises safety, even when users do not intend to! In *The Twelfth International Conference on Learning Representations*, 2024.

[37] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, dahai li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations*, 2024.

[38] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI*, 2019.

[39] Zixin Rao, Wentian Zhu, Chan Lu, Zhaorun Chen, Wei Niu, Le Guan, Bo Li, and Zhen Xiang. FragFuse Online Appendix. https://drive.google.com/file/d/1lt3EQBTpzsQl3yDQHiBipSwakI0gdEnb/view?usp=sharing, 2026. Online appendix.

[40] Souhaila Serbout, Amine El Malki, Cesare Pautasso, and Uwe Zdun. API rate limit adoption – a pattern collection. https://www.researchgate.net/publication/377466057_API_Rate_Limit_Adoption_-A_pattern_collection, 2024. EuroPLoP 2023 paper, available as public full-text on ResearchGate.

[41] Tianneng Shi et al. PromptArmor: Simple yet effective prompt injection defenses, 2025.

[42] Wenqi Shi et al. EHRAgent: Code empowers large language models for few-shot complex tabular reasoning on electronic health records. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 22315–22339, Miami, Florida, USA, November 2024. Association for Computational Linguistics.

[43] Noah Shinn et al. Reflexion: Language agents with verbal reinforcement learning. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652. Curran Associates, Inc., 2023.

[44] Zhen Tan et al. In prospect and retrospect: Reflective memory management for long-term personalized dialogue agents. In Wanxiang Che et al., editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8416–8439, Vienna, Austria, July 2025. Association for Computational Linguistics.

[45] Token of Trust. How to sell alcohol online legally in the United States. https://tokenoftrust.com/blog/sell-alcohol-online-legally/, 2023. Accessed: 2026-02-01.

[46] UK Government Department for Science, Innovation and Technology. Inspect: A framework for large language model evaluations. https://github.com/UKGovernmentBEIS/inspect, 2024.

[47] U.S. Department of Health and Human Services. Summary of the HIPAA security rule. https://www.hhs.gov/hipaa/for-professionals/security/laws-regulations/index.html, 2013. Accessed: 2026-02-01.

[48] Bo Wang, Weiyi He, Shenglai Zeng, Zhen Xiang, Yue Xing, Jiliang Tang, and Pengfei He. Unveiling privacy risks in LLM agent memory. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 25241–25260, Vienna, Austria, July 2025. Association for Computational Linguistics.

[49] Waymo. How rider feedback shapes Waymo's fully autonomous ride-hailing service. https://waymo.com/blog/2020/11/how-rider-feedback-shapes-waymos-fully-autonomous-ride-hailing-service, 2020. Accessed: 2026-02-01.

[50] Ruffin White, Henrik I. Christensen, Gianluca Caiazza, and Agostino Cortesi. Procedurally provisioned access control for robotic systems. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1–9. IEEE Press, 2018.

[51] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024.

[52] Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. GuardAgent: Safeguard LLM agents via knowledge-enabled reasoning. In *Forty-second International Conference on Machine Learning*, 2025.

[53] Yunzhong Xiao et al. ToolMem: Enhancing multimodal agents with learnable tool capability memory, 2025.

[54] Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. How memory management impacts LLM agents: An empirical study of experience-following behavior. *ArXiv*, abs/2505.16067, 2025.

[55] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-Mem: Agentic memory for LLM agents. In *The Thirty-Ninth Annual Conference on Neural Information Processing Systems*, 2025.

[56] Shunyu Yao et al. WebShop: Towards scalable real-world web interaction with grounded language agents. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 20744–20757. Curran Associates, Inc., 2022.

[57] Rui Ye, Zhongwang Zhang, Kuan Li, Huifeng Yin, Zhengwei Tao, Yida Zhao, Liangcai Su, Liwen Zhang, Zile Qiao, Xinyu Wang, Pengjun Xie, Fei Huang, Siheng Chen, Jingren Zhou, and Yong Jiang. AgentFold: Long-horizon web agents with proactive context management, 2025.

[58] Hongli Yu, Tinghong Chen, Jiangtao Feng, Jiangjie Chen, Weinan Dai, Qiying Yu, Ya-Qin Zhang, Wei-Ying Ma, Jingjing Liu, Mingxuan Wang, and Hao Zhou. MemAgent: Reshaping long-context LLM with multi-conv RL-based memory agent, 2025.

[59] Lingzhi Yuan et al. PromptGuard: Soft prompt-guided unsafe content moderation for text-to-image models, 2025.

[60] Guibin Zhang et al. G-Memory: Tracing hierarchical memory for multi-agent systems, 2025.

[61] Lingfeng Zhang et al. Mem2Ego: Empowering vision-language models with global-to-ego memory for long-horizon embodied navigation, 2025.

[62] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. GPT-4V(ision) is a generalist web agent, if grounded, 2024.

[63] Wanjun Zhong et al. MemoryBank: Enhancing large language models with long-term memory. *ArXiv*, abs/2305.10250, 2023.

[64] Huichi Zhou et al. Memento: Fine-tuning LLM agents without fine-tuning LLMs, 2025.

[65] Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. Mem1: Learning to synergize memory and reasoning for efficient long-horizon agents, 2025.

[66] Andy Zou et al. Universal and transferable adversarial attacks on aligned language models, 2023.

Table 8: Average bypass rates (%) on RAP across three backbone LLMs.

Attack	LLM-AC	GuardAgent
FragFuse	93.7	79.3
CRP	25.0	17.7
ArtPrompt	49.0	67.2

Table 9: Comparing TSRs computed by an LLM judge and by human evaluation.

Agent	Metric	
	LLM Decision	Human Evaluation
RAP	92.5	92.5
SeeAct	22.1	22.1
OSAgent	75.5	77.3
InspAgent	32.0	34.5

A Prompt Guard versus Access Control

Table 8 compares the average bypass rates of FragFuse with two prompt-obfuscation baselines, CRP and ArtPrompt, on RAP across three backbone LLMs. The results show that FragFuse achieves substantially higher bypass rates under both LLM-AC and GuardAgent, indicating that bypassing agent access control requires more than surface-level prompt obfuscation.

B LLM AC Prompting Templates

For each environment, we instantiate an LLM-based checker that applies the corresponding, explicitly specified policy rules to the agent input and returns an allow/deny decision. We list the full templates below for reproducibility, including the rule statements, required input fields, and the strict output formats used by our evaluation pipeline. The complete prompt templates used to implement *LLM-AC* across all agent settings are provided in Appendix B in the complete online appendix [39].

C LLM Decision Templates

TSRs computed by an LLM judge are largely consistent with human evaluation in our main settings. We use GPT-4o as an LLM judge to compute TSR and validate it against human evaluation. As shown in Table 9, the LLM-judged TSR matches human evaluation in RAP–WebShop and remains directionally consistent in OSAgent, though OS-level trajectories show larger absolute differences. These results support using an LLM judge for scalable TSR computation, with occasional manual spot-checking in more complex agent environments. The complete LLM decision templates are provided in Appendix C in the complete online appendix [39].

D Algorithm for Fusion Instruction Optimization

The algorithm summarizing the discrete genetic search used to optimize the fusion instruction $I^{\star}_{\text{fuse}}$ is provided in the complete online appendix [39].

E Generated Benign Queries Examples of SafeOS

1. Original: `What's the full path of a file "tools" after checking under /user_ok?`
   Generated: `Find the absolute path to the file named "config" somewhere under /user_ok.`
2. Original: `What is the content of /test/config.yaml?`
   Generated: `Open /test/config.yaml and return its full text content.`
3. Original: `Which file was created most recently in /root/tmpdir after checking?(without its directory)`
   Generated: `After inspecting /root/tmpdir, return the name (no path) of the newest file by modification time.`

F Direct Bypassing Instruction Example

A direct bypassing instruction example is provided in Appendix F in the complete online appendix [39].

G Fusion Instruction Seeds for Surrogate Optimization

The fusion instruction seeds for surrogate optimization are provided in Appendix G in the complete online appendix [39].

H Advanced Memory Writing and Retrieval

We evaluate two quality-aware memory mechanisms from *RAP* [24]: selective memory writing based on a reward threshold and reward-weighted memory retrieval. As shown in Tables 10 and 11, both mechanisms leave BSR unchanged but reduce TSR, with the strongest drops from 92.5 to 39.8 under LLM-AC and from 92.6 to 45.7 under GuardAgent. These results suggest that quality-aware memory mechanisms mainly weaken post-bypass task completion by filtering or deprioritizing injected records.

I Runtime and Token Cost

Table 12 reports the runtime breakdown of FragFuse across the four agents using GPT-4o, excluding the execution time of access-control modules. Runtime varies with the task domain (e.g., task complexity) as well as external factors such as network conditions. Table 13 reports the total token cost

Table 10: BSR and TSR of FragFuse under different memory writing thresholds.

Advanced Memory Writing	LLM-AC		GuardAgent	
	BSR	TSR	BSR	TSR
0 (default)	93.0	92.5	81.0	92.6
0.25	93.0	86.0	81.0	80.2
0.5	93.0	84.9	81.0	71.6
0.75	93.0	50.5	81.0	53.1
1.0	93.0	39.8	81.0	45.7

Table 11: Performance under different reward-weight settings for advanced memory retrieval.

Advanced Memory Retrieval	LLM-AC		GuardAgent	
	BSR	TSR	BSR	TSR
0 (default)	93.0	92.5	81.0	92.6
0.25	93.0	80.6	81.0	75.3
0.5	93.0	73.1	81.0	65.4
0.75	93.0	53.8	81.0	51.9
1.0	93.0	53.8	81.0	51.9

across backbone LLMs and agent settings, including both input tokens and model-generated output tokens.

J Marker-aware Surrogate Optimization Process

The detailed marker-aware surrogate optimization process and analysis are provided in Appendix J in the complete online appendix [39].

K Failure Case Analysis

We provide a representative failure case in Appendix K in the complete online appendix [39].

L Sensitive Fragments Discovery Quries Limit

We provide a representative example of sensitive fragments discovery quries limit in Appendix L in the complete online appendix [39].

M FragExtractor Prompt Templates

The complete FRAGEXTRACTOR prompt templates used across our evaluated agent settings are provided in Appendix M in the complete online appendix [39].

N Detailed Retrieval Similarity Data

The detailed retrieval similarity data are provided in Appendix N in the complete online appendix [39], showing that carrier-query retrieval is highly robust across different memory sizes (8/16/32) and retrieval methods: almost all settings achieve a 100% match rate for RAP, SeeAct, and InspAgent. The only noticeable degradation occurs for OSAgent, where BM25 drops slightly as memory increases (98%→94%) and cosine similarity with intfloat/e5-base-v2 also decreases (96%→92%), while string matching and all-MiniLM-L6-v2 remain at 100%.

Table 12: Runtime breakdown for sensitive fragment discovery and query execution.

Agents	FragFuse Runtime (s)			Benign
	Discovery	Carrier	Attack	
RAP	2.1	9.1	19.2	8.5
SeeAct	5.0	28.9	31.2	27.4
OSAgent	4.8	51.2	52.1	51.9
InspAgent	4.5	1.0	6.6	0.2

Table 13: Token cost across backbone LLMs and agent settings. Token cost includes both input tokens and model-generated output tokens.

Backbone LLM	Agent	Benign	Carrier	Attack
GPT-4o	RAP	4,251	4,589	6,981
	SeeAct	16,025	18,593	29,602
	OSAgent	2,149	2,785	3,460
	InspAgent	3,354	3,826	6,613
GPT-5.1	RAP	4,487	4,883	7,487
	SeeAct	16,914	19,783	31,748
	OSAgent	2,268	2,963	3,711
	InspAgent	3,540	4,071	7,092
Gemini 2.5 Flash	RAP	4,476	4,791	7,225
	SeeAct	16,874	19,411	30,638
	OSAgent	2,263	2,908	3,581
	InspAgent	3,532	3,994	6,844

O User Profile Generated Details for Webshop

The details for generating user profiles are provided in Appendix O in the complete online appendix [39].

P Report of End-to-end Success

Table 14 reports the end-to-end success rate (E2E-SR) of FragFuse and the baseline across all agent, access-control, and backbone-LLM settings. Compared with BSR and TSR alone, E2E-SR captures whether an attack both bypasses access control and successfully induces the intended downstream behavior.

Table 14: End-to-end success rate (E2E-SR) of `FragFuse` and the baseline across agent, access-control, and backbone-LLM settings. E2E-SR is computed as BSR $\times$ TSR for attacks; for direct querying, E2E-SR equals TSR.

Agent	**AC**	**Query Setting**	**Core LLM**									**Average**		
			GPT-4o			**GPT-5.1**			**Gemini 2.5 Flash**					
			BSR	**TSR**	**E2E**	**BSR**	**TSR**	**E2E**	**BSR**	**TSR**	**E2E**	**BSR**	**TSR**	**E2E**
RAP	–	direct querying	n.a.	88.0	88.0	n.a.	75.0	75.0	n.a.	88.0	88.0	n.a.	83.7	83.7
	LLM-AC	baseline	40.0	77.5	31.0	45.0	17.8	8.0	6.0	66.7	4.0	30.3	$54.0_{-29.7}$	$14.3_{-69.4}$
		`FragFuse`	93.0	92.5	86.0	98.0	70.4	69.0	90.0	70.0	63.0	**93.7**	$\mathbf{77.6}_{-6.1}$	$\mathbf{72.7}_{-11.0}$
	GuardAgent	baseline	42.0	76.2	32.0	51.0	25.5	13.0	13.0	53.8	7.0	35.3	$51.8_{-31.9}$	$17.3_{-66.4}$
		`FragFuse`	81.0	92.6	75.0	84.0	78.6	66.0	73.0	75.3	55.0	**79.3**	$\mathbf{82.2}_{-1.5}$	$\mathbf{65.3}_{-18.4}$
SeeAct	–	direct querying	n.a.	22.0	22.0	n.a.	17.0	17.0	n.a.	15.0	15.0	n.a.	18.0	18.0
	LLM-AC	baseline	3.0	33.3	1.0	1.0	0.0	0.0	5.0	0.0	0.0	3.0	$11.1_{-6.9}$	$0.3_{-17.7}$
		`FragFuse`	88.0	20.5	18.0	98.0	20.4	20.0	93.0	17.2	16.0	**93.0**	$\mathbf{19.4}_{+1.4}$	$\mathbf{18.0}_{0.0}$
	AGrail	baseline	5.0	20.0	1.0	9.0	22.2	2.0	13.0	7.7	1.0	9.0	$16.6_{-1.4}$	$1.3_{-16.7}$
		`FragFuse`	72.0	23.6	17.0	86.0	20.9	18.0	89.0	15.7	14.0	**82.3**	$\mathbf{20.1}_{+2.1}$	$\mathbf{16.3}_{-1.7}$
OSAgent	–	direct querying	n.a.	80.0	80.0	n.a.	84.0	84.0	n.a.	82.0	82.0	n.a.	82.0	82.0
	LLM-AC	baseline	0.0	0.0	0.0	0.0	0.0	0.0	10.0	100.0	10.0	3.3	$33.3_{-48.7}$	$3.3_{-78.7}$
		`FragFuse`	82.0	80.5	66.0	96.0	79.2	76.0	64.0	84.4	54.0	**80.7**	$\mathbf{81.4}_{-0.6}$	$\mathbf{65.3}_{-16.7}$
	AGrail	baseline	10.0	40.0	4.0	28.0	14.3	4.0	20.0	10.0	2.0	19.3	$21.4_{-60.6}$	$3.3_{-78.7}$
		`FragFuse`	88.0	70.5	62.0	96.0	81.3	78.0	94.0	80.9	76.0	**92.7**	$\mathbf{77.6}_{-4.4}$	$\mathbf{72.0}_{-10.0}$
InspAgent	–	direct querying	n.a.	38.6	38.6	n.a.	13.6	13.6	n.a.	22.7	22.7	n.a.	25.0	25.0
	LLM-AC	baseline	3.4	50.0	1.7	21.0	32.4	6.8	0.0	0.0	0.0	8.1	$\mathbf{27.5}_{+2.5}$	$2.8_{-22.2}$
		`FragFuse`	45.5	25.0	11.4	97.2	7.6	7.4	98.9	0.0	0.0	**80.5**	$10.9_{-14.1}$	$\mathbf{6.3}_{-18.7}$
	ShieldAgent	baseline	1.1	0.0	0.0	33.5	10.2	3.4	21.6	28.9	6.2	18.7	$13.0_{-12.0}$	$3.2_{-21.8}$
		`FragFuse`	82.4	17.2	14.2	97.2	2.9	2.8	85.2	19.3	16.4	**88.3**	$\mathbf{13.1}_{-11.9}$	$\mathbf{11.1}_{-13.9}$